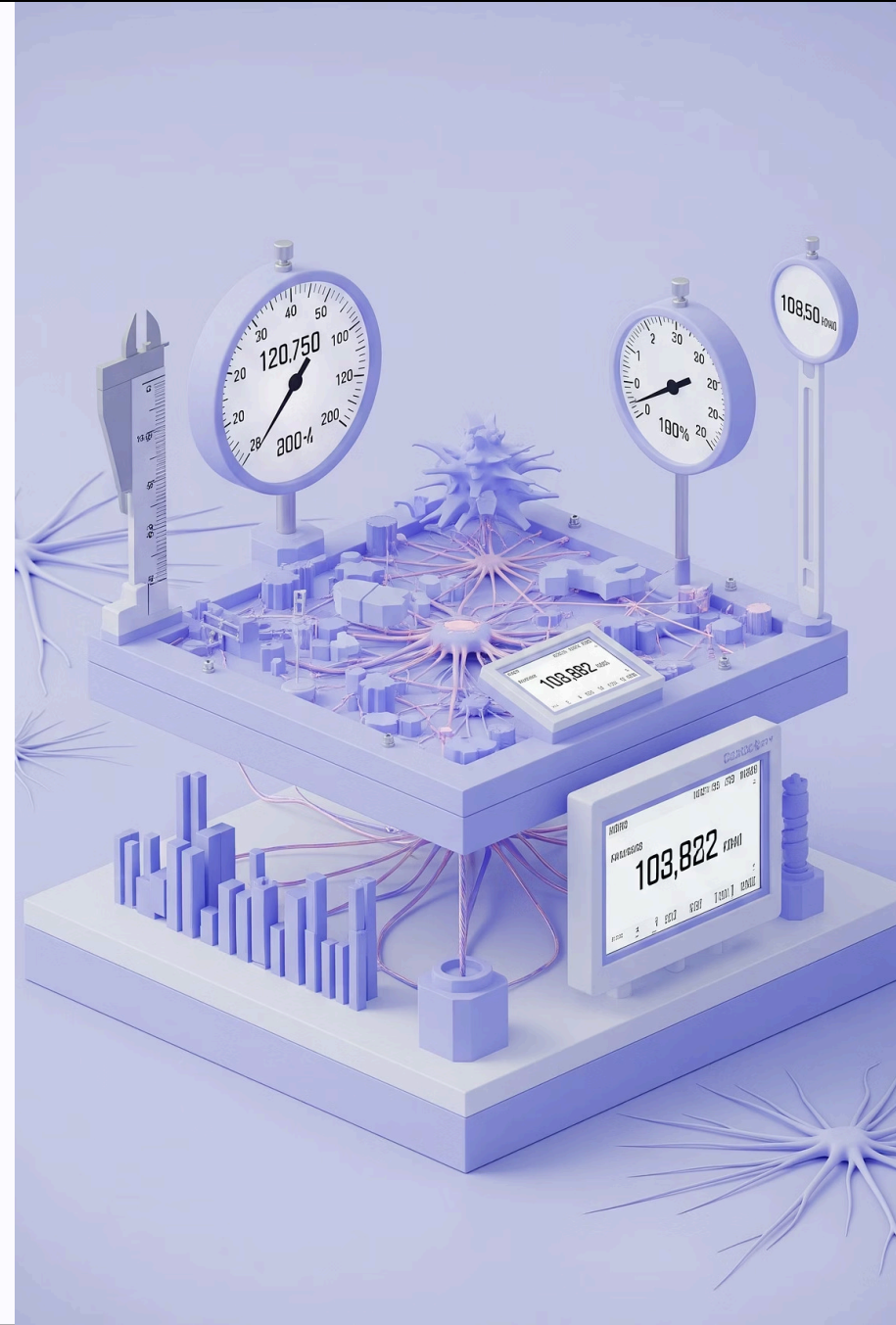


Benchmarking Optimized Models

Measuring Speed, Cost, and Quality After Optimization

A comprehensive guide for ML engineers to systematically evaluate model performance after optimization techniques have been applied.



Why Benchmark?



Prove Performance Gains

Optimizations change behavior. You need concrete evidence of **latency improvements, throughput gains, memory reductions, and cost savings.**



Validate Quality

Ensure **accuracy & fairness** are not degraded by your optimization efforts.



Deployment Confidence

Benchmarking provides the data-driven confidence needed before pushing optimized models to production environments.

Key Metrics to Measure



Latency

p50, p95, p99 response times

Throughput

Requests/sec or tokens/sec

Memory Footprint

GPU/CPU peak usage, model size on disk

Energy Efficiency

Joules per inference, \$/1K predictions

Task Quality

Accuracy, BLEU, ROUGE, F1 scores

Benchmarking Workflow

Define Baseline

Establish performance metrics of the original unoptimized model as your reference point.

Apply Optimization

Implement techniques like quantization, pruning, distillation, or sparsity.

Measure Infrastructure Metrics

Evaluate latency, throughput, memory usage, and energy consumption.

Measure Task Metrics

Assess accuracy, fairness, robustness across your evaluation datasets.

Compare & Decide

Analyze results against baseline to accept or reject the optimized model.

Tools for Benchmarking

Runtime Performance

- ONNX Runtime
- TensorRT
- TVM
- OpenVINO

LLM Optimization

- DeepSpeed
- FSDP
- Hugging Face Optimum

Standardized Benchmarks

- TorchBench
- MLPerf

Custom Testing

- Python's `timeit`
- Apache Benchmark (ab)
- Locust for load testing

The right tools depend on your model type, optimization approach, and deployment target.

Example: PyTorch Benchmark

```
import torch, time

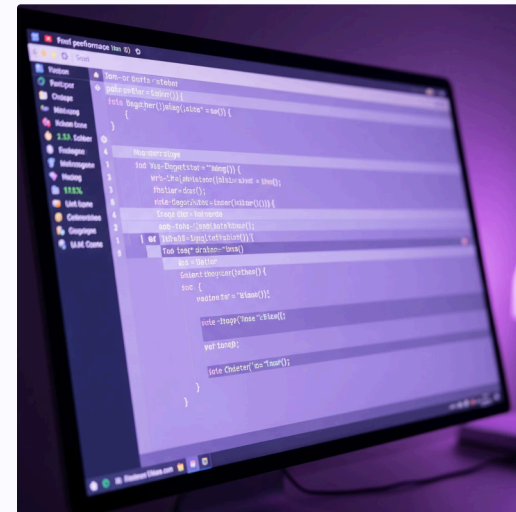
model.eval()
inputs = torch.randn(1, 3, 224, 224).to("cuda")

start = time.time()
for _ in range(100):
    with torch.no_grad():
        _ = model(inputs)
    torch.cuda.synchronize()
end = time.time()

print("Avg latency (ms):", (end-start)/100*1000)
```

Key implementation notes:

- Always use `model.eval()` for inference benchmarking
- Include `torch.no_grad()` to mimic production
- `torch.cuda.synchronize()` ensures GPU operations complete
- Run multiple iterations (warm-up + measurement)
- Use realistic input shapes/types



Comparing Baseline vs Optimized

Metric	Baseline FP32	Optimized INT8	Delta
Latency (ms)	45	18	-60%
Throughput	100 req/s	280 req/s	+180%
Accuracy	92.1%	91.7%	-0.4%
Memory (GB)	4.8	2.1	-56%

This example shows how INT8 quantization dramatically improves performance metrics with minimal accuracy impact - a successful optimization.

Tail Latency Matters

Don't just measure average (p50) performance. **Tail latencies** at p95 and p99 can significantly impact user experience.

Real-time Systems

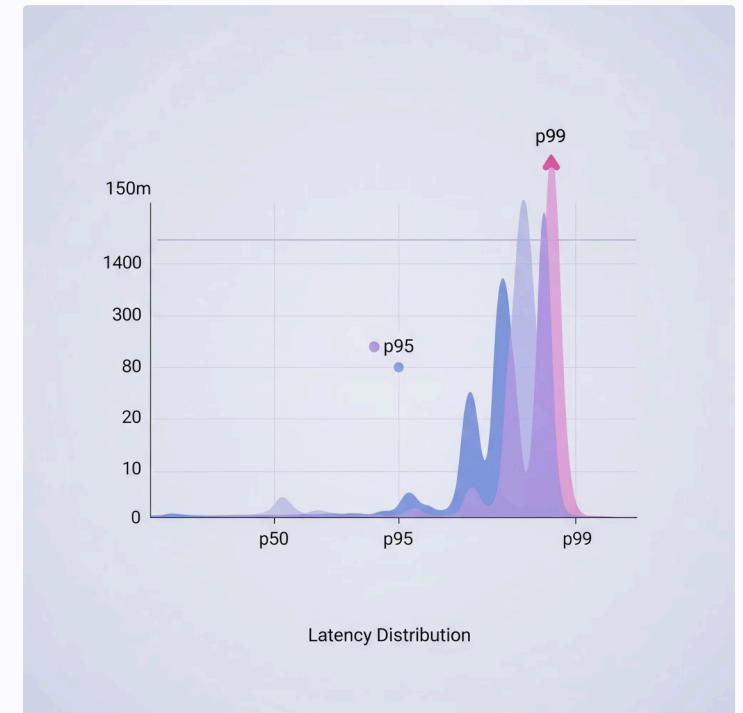
Chatbots, recommendation systems, and voice assistants are particularly sensitive to latency spikes.

Under Load

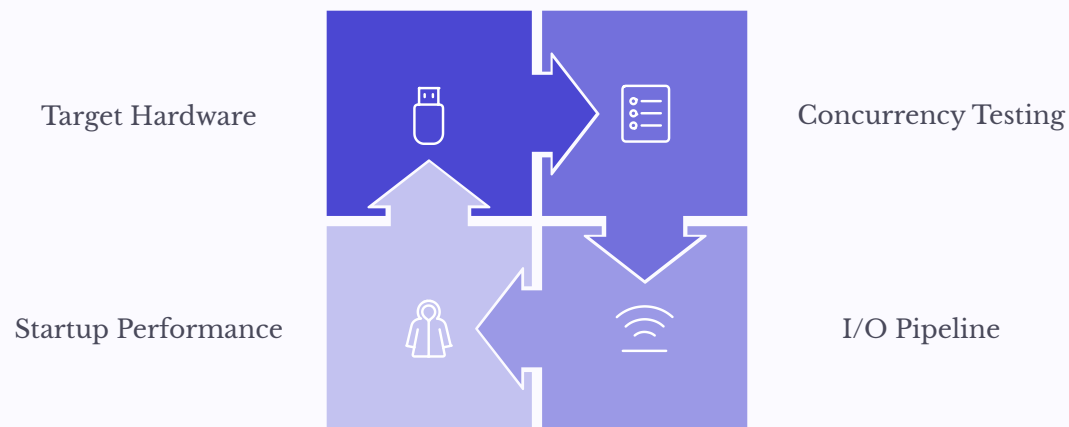
Always benchmark with realistic concurrent requests to reveal performance degradation patterns.

SLA Compliance

Service Level Agreements often specify maximum acceptable p99 latency, not just averages.



Deployment Considerations



Target Hardware

Always benchmark on your actual deployment hardware (GPU vs CPU vs Edge TPU). Performance characteristics vary dramatically.

End-to-End Pipeline

Include tokenization, preprocessing, batching, and postprocessing in your benchmarks to capture real-world performance.

Remember: **Cold start** latency can be 5-10x higher than **warm start** latency for large models. This is critical for serverless deployments.

Best Practices



Automate

Integrate benchmarking into CI/CD pipeline to catch performance regressions early.



Multiple Workloads

Compare batch processing vs. streaming inference scenarios.



Holistic Measurement

Always measure both infrastructure metrics AND task quality together.



A/B Testing

Use canary deployments to validate optimized models in production environments.



Key Takeaways

1 Optimization Requires Validation

Optimization doesn't equal success without comprehensive benchmarking.

2 Dual Objectives

You must prove both efficiency gains AND quality retention.

3 End-to-End Perspective

Benchmark the complete pipeline from data input to final output.

4 Deployment Confidence

Thorough benchmarking enables **safe, efficient deployment** of optimized models.



Questions?